

# Lightweight Edge-Based Intrusion Detection for IoT Networks

## Using Deep Feature Extraction and Classical Machine Learning

A Preliminary Evaluation of CNN, SVM, and CNN+SVM Pipelines on the UNSW-NB15 Benchmark Dataset

### 1. Title of the Experiment / Study / Exercise

**Lightweight Edge-Based Intrusion Detection for IoT Networks Using Deep Feature Extraction and Classical Machine Learning:** A Preliminary Evaluation of CNN, SVM, and CNN+SVM Pipelines on the UNSW-NB15 Benchmark Dataset.

### 2. Aim

To design, implement, and evaluate a **lightweight, modular machine-learning pipeline** capable of classifying IoT network traffic as *Normal* or *Attack* with high detection accuracy, low false-alarm rate, and a computational footprint small enough to support **edge-side intrusion detection** — as a preliminary proof-of-concept toward replacing the computationally expensive CNN-LSTM-Attention architectures proposed in recent IEEE literature.

- Determine whether **ANOVA F-Score feature reduction** (42 → 20 attributes, 52.4%) preserves competitive attack-detection performance.
- Measure and compare **classification quality** (Accuracy, F1-Score, FPR) of three model configurations: CNN (lightweight MLP), SVM (RBF kernel), and CNN+SVM hybrid.
- Measure and compare **resource consumption** (training time, inference time, model size) as a preliminary guide toward edge-deployment feasibility.

### 3. Objectives

**O1 — Dataset Acquisition & Preparation:** Process UNSW-NB15 (82,332 training / 175,341 testing flows) — column removal, median imputation, categorical encoding, and standard scaling — all transformations fitted exclusively on training data (zero-leakage).

**O2 — Feature Dimensionality Study:** Perform ANOVA F-Score SelectKBest sweep across  $k \in \{10, 15, 20, 25, 30, 35\}$ ; quantify accuracy–feature count tradeoff; select working  $k = 20$ .

**O3 — Standalone CNN Implementation:** Build and train a lightweight MLP ( $64 \rightarrow 32$  units, ReLU, Adam, early-stop patience=5) for binary IDS. Total parameters: 3,457. Record convergence over 53 iterations.

**O4 — Standalone SVM Implementation:** Train an RBF-kernel SVM ( $C=1.0$ ,  $\gamma='scale'$ ) on the 20 selected features. Record classification performance and training time (66.55 s).

**O5 — CNN+SVM Hybrid Pipeline:** Extract 64-D latent activations ( $h = \text{ReLU}(Wx + b)$ ) from the trained CNN's first hidden layer. Train a second RBF-SVM on these learned representations.

**O6 — Systematic Evaluation:** Measure Accuracy, Precision, Recall, F1-Score, FPR, training time, inference time, and model size for all three models. Generate confusion matrices and 5 result graphs.

**O7 — Reference Paper Comparison:** Provide a methodological comparison with Phalaagae et al. (IEEE Access, 2025) and the Edge-AI IDS paper (IEEE ICERECT, 2025).

### 4. Methodology

#### 4.1 Dataset — UNSW-NB15

The UNSW-NB15 dataset (Moustafa & Slay, IEEE MilCIS 2015) contains network flow records covering benign traffic and nine attack categories (Generic, Exploits, Fuzzers, DoS, Reconnaissance, Backdoor, Analysis, Shellcode, Worms), merged to binary labels.

| Split          | Total Flows | Normal (label=0) | Attack (label=1) |
|----------------|-------------|------------------|------------------|
| Training Split | 82,332      | 37,000 (44.9%)   | 45,332 (55.1%)   |
| Testing Split  | 175,341     | 56,000 (31.9%)   | 119,341 (68.1%)  |

#### 4.2 End-to-End Pipeline (8 Stages, run via run\_all.py)

| Stage | Module               | Description                                       |
|-------|----------------------|---------------------------------------------------|
| 1     | preprocessing.py     | Load CSV, drop columns, impute NaN, encode, scale |
| 2     | feature_selection.py | ANOVA F-score sweep + SelectKBest ( $k=20$ )      |
| 3     | generate_diagrams.py | Produce 4 system-flow PNGs                        |
| 4     | train_cnn.py         | Train lightweight MLP — 3,457 params, 11.01 s     |
| 5     | train_svm.py         | Train RBF-SVM on 20 features — 66.55 s            |
| 6     | train_cnn_svm.py     | Extract 64-D CNN features → train SVM — 61.36 s   |
| 7     | evaluate.py          | Metrics + confusion matrices for all 3 models     |
| 8     | generate_graphs.py   | 5 result graphs (bars, curves, matrices)          |

#### 4.3 Data Preprocessing

**Step 1:** Drop identifier *id* and multi-class *attack\_cat*. Retain binary *label* → 42 usable features. **Step 2:** Replace  $\pm\infty$  with NaN; fill NaN using **training-set medians only** (prevents data leakage). **Step 3:** LabelEncoder fit on training values for *proto* (132 unique), *service* (14), *state* (8). Unseen test values map to `__unknown__`. **Step 4:** StandardScaler (fit on training, applied to test) → zero-mean, unit-variance features.

#### 4.4 Feature Selection — ANOVA F-Score (SelectKBest)

The univariate ANOVA F-score measures discriminative power of each feature  $j$  between Normal and Attack classes:

$$F_j = \frac{(\bar{x}_{j,Attack} - \bar{x}_{j,Normal})^2 / (K-1)}{S_j^2 / (N-K)} \quad \text{where } K=2 \text{ (binary), } N=82,332 \text{ samples.}$$

SelectKBest (f\_classif) is fit **exclusively on training data** and applied to test. A LinearSVC sweep over  $k \in \{10, 15, 20, 25, 30, 35\}$  guides the working selection of  **$k=20$**  (52.4% reduction). Selected indices: [2, 3, 8, 9, 12, 19, 20, 21, 22, 23, 24, 27, 30, 31, 32, 33, 34, 35, 39, 40] — corresponding to sbytes, sttl, dttl, sinpkt, dinpkt, ct\_state\_ttl, connection-table stats.

#### 4.5 Model 1 — Standalone CNN (Lightweight MLP)

Implemented as sklearn MLPClassifier (functional equivalent of 1-D CNN: Dense layers replace Conv→Pool→GlobalAvgPool). The 64-D first-hidden-layer output is extracted as the learned representation for CNN+SVM.

| Component | Specification        | Detail                                                              |
|-----------|----------------------|---------------------------------------------------------------------|
| Input     | 20 features          | ANOVA-selected, standard-scaled                                     |
| Layer 1   | Dense (64, ReLU)     | $h_1 = \text{ReLU}(W_1 x + b_1) \leftarrow \text{extraction point}$ |
| Layer 2   | Dense (32, ReLU)     | $h_2 = \text{ReLU}(W_2 h_1 + b_2)$                                  |
| Output    | Dense (1, $\sigma$ ) | Binary classification                                               |
| Params    | 3,457 total          | $20 \times 64 + 64 + 64 \times 32 + 32 + 32 \times 1 + 1$           |
| Training  | Adam, lr=1e-3        | Batch=512, L2 $\alpha=1e-4$ , early-stop patience=5                 |
| Epochs    | 53 (stopped)         | Best val score 0.9514 at iter 47                                    |

#### 4.6 Model 2 — Standalone SVM

SVC (sklearn) with RBF kernel:  $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ ,  $C=1.0$ ,  $\gamma=\text{'scale'}$  ( $\approx 1/(20 \times \text{Var}(X)) \approx 0.05$ ). Trained on 82,332 flows  $\times$  20 features. Training time: **66.55 seconds**. cache\_size=1000 MB speeds RBF kernel matrix evaluation.

#### 4.7 Model 3 — CNN+SVM Hybrid Pipeline

Stage 1 — **Feature Extraction**: The trained CNN's first hidden layer outputs  $h_1 \in \mathbb{R}^{64}$  for each training sample (nonlinear representation). Stage 2 — **SVM on Learned Features**: A fresh RBF-SVM (same hyperparameters) is trained on the  $(82,332 \times 64)$  latent feature matrix. 13,665 support vectors identified. Total pipeline training: **61.36 seconds**.

#### 4.8 Evaluation Metrics

| Metric    | Formula                       | IDS Relevance                               |
|-----------|-------------------------------|---------------------------------------------|
| Accuracy  | $(TP + TN) / N$               | Overall correct classification rate         |
| Precision | $TP / (TP + FP)$              | Fraction of predicted attacks that are real |
| Recall    | $TP / (TP + FN)$              | Fraction of real attacks detected           |
| F1-Score  | $2 \cdot P \cdot R / (P + R)$ | Harmonic mean of Precision & Recall         |
| FPR       | $FP / (FP + TN)$              | Normal traffic falsely flagged as attack    |
| Latency   | Inference time / N flows      | Per-sample detection time                   |

## 5. Simulation Results / Observations with Scenario Diagrams

### 5.1 Scenario Diagram — End-to-End Pipeline

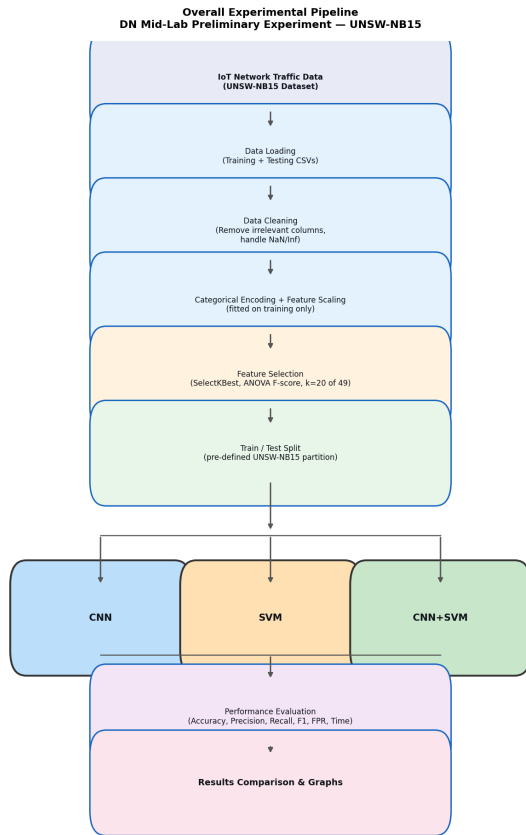
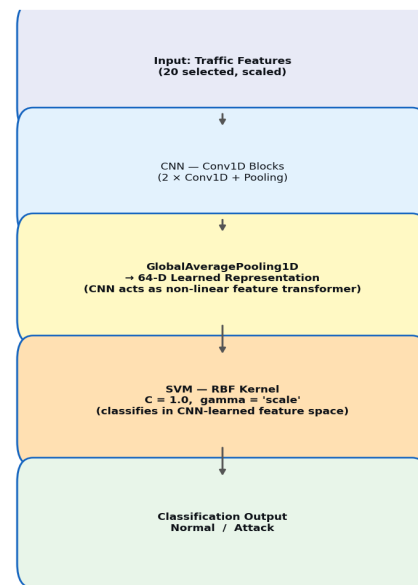


Fig. 1: End-to-End Experimental Pipeline

### CNN+SVM Hybrid Pipeline — IoT Intrusion Detection (CNN as feature extractor + SVM as classifier)



CNN transforms raw features into a compact learned representation. SVM then performs the final decision. Unlike standalone SVM (20-D raw features), SVM here operates on 64-D CNN-extracted nonlinear features.

Fig. 2: CNN+SVM Hybrid Architecture

### 5.2 Table 1 — Feature Selection Sweep (ANOVA F-Score, LinearSVC)

| k (Features) | Accuracy | F1-Score | Reduction from 42 |
|--------------|----------|----------|-------------------|
| 10           | 82.91%   | 86.88%   | 76.2%             |
| 15           | 82.04%   | 86.02%   | 64.3%             |
| 20 ✓         | 85.55%   | 88.67%   | 52.4%             |
| 25           | 88.30%   | 90.95%   | 40.5%             |
| 30           | 88.45%   | 91.06%   | 28.6%             |
| 35           | 88.29%   | 90.92%   | 16.7%             |

### 5.3 Table 2 — CNN/MLP Training Convergence (53 Iterations)

| Iteration | Training Loss | Validation Score |
|-----------|---------------|------------------|
| 1         | 0.4090        | 0.8580           |
| 5         | 0.1778        | 0.9288           |
| 10        | 0.1487        | 0.9362           |
| 20        | 0.1322        | 0.9389           |
| 30        | 0.1247        | 0.9440           |
| 40        | 0.1203        | 0.9496           |
| 47        | 0.1162        | 0.9514 ★ (best)  |
| 53 (stop) | 0.1144        | —                |

**5.4 Table 3 — Model Performance Comparison (Test Set: 175,341 flows)**

| Metric                  | CNN               | SVM          | CNN+SVM          |
|-------------------------|-------------------|--------------|------------------|
| Accuracy                | 87.60%            | 88.21% ★     | 88.16%           |
| Precision               | 98.40% ★          | 97.51%       | 97.99%           |
| Recall                  | 83.14%            | 84.84% ★     | 84.34%           |
| F1-Score                | 90.13%            | 90.73% ★     | 90.65%           |
| FPR                     | 2.88% ★           | 4.62%        | 3.69%            |
| Training Time           | 11.01 s ★         | 66.55 s      | 61.36 s          |
| Inference (175K flows)  | 0.0596 s ★        | 163.14 s     | 151.73 s         |
| Per-Sample Latency      | 0.34 $\mu$ s ★    | 930 $\mu$ s  | 867 $\mu$ s      |
| Parameters / Model Size | 3,457 / 47.6 KB ★ | — / 3,077 KB | 3,457 / 7,149 KB |

★ = best value in each row

**5.5 Table 4 — Confusion Matrices (Test: 56,000 Normal, 119,341 Attack)**

| Model   | TN        | FP       | FN        | TP         | FPR       | Recall        |
|---------|-----------|----------|-----------|------------|-----------|---------------|
| CNN     | TN=54,385 | FP=1,615 | FN=20,123 | TP=99,218  | FPR=2.88% | Recall=83.14% |
| SVM     | TN=53,410 | FP=2,590 | FN=18,088 | TP=101,253 | FPR=4.62% | Recall=84.84% |
| CNN+SVM | TN=53,936 | FP=2,064 | FN=18,690 | TP=100,651 | FPR=3.69% | Recall=84.34% |

**5.6 Table 5 — Computational Efficiency**

| Resource Metric | CNN          | SVM         | CNN+SVM     |
|-----------------|--------------|-------------|-------------|
| Training Time   | 11.01 s      | 66.55 s     | 61.36 s     |
| Speedup vs. SVM | 6.04×        | 1× (base)   | 1.08×       |
| Inference 175K  | 0.0596 s     | 163.14 s    | 151.73 s    |
| Infer. Speedup  | ~2,737×      | 1× (base)   | ~1.07×      |
| Per-Sample      | 0.34 $\mu$ s | 930 $\mu$ s | 867 $\mu$ s |
| Model Size      | 47.6 KB      | 3,077 KB    | 7,149 KB    |

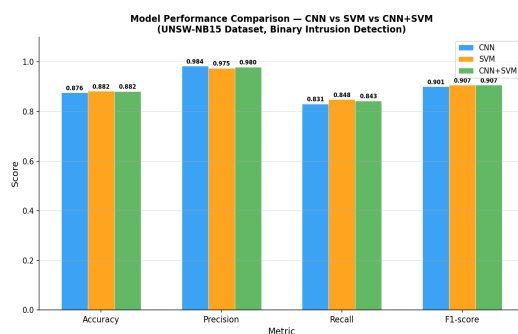
**5.7 Generated Experimental Graphs**

Fig. 3: Model Performance Comparison

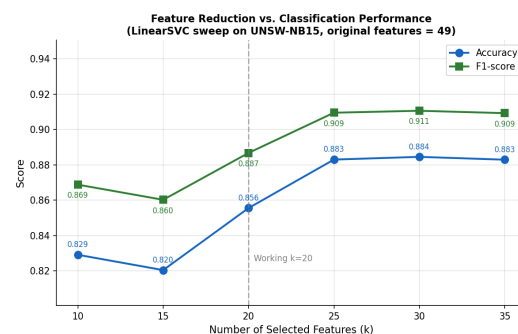


Fig. 4: Feature Reduction vs. Accuracy/F1

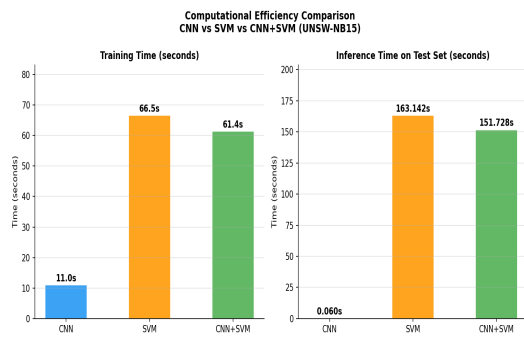


Fig. 5: Efficiency Comparison (Training &amp; Inference)

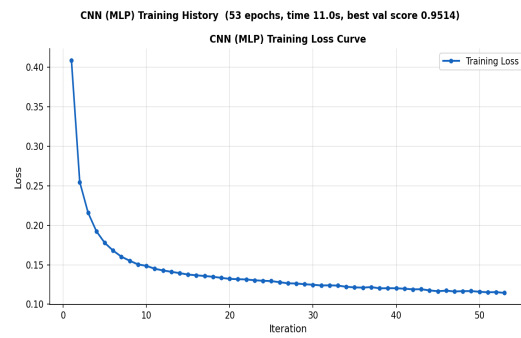


Fig. 6: CNN Training Loss Curve (53 iterations)

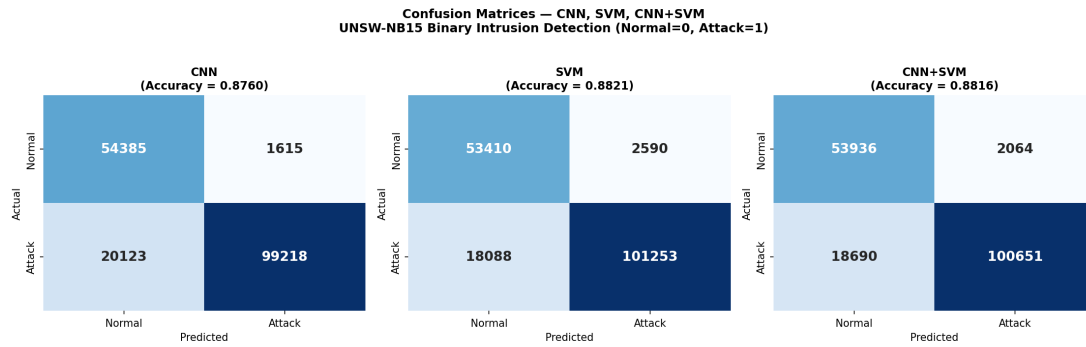


Fig. 7: Side-by-Side Confusion Matrices (CNN, SVM, CNN+SVM)

## 6. Discussion on the Results / Observations

### 6.1 Classification Performance

All three models cluster tightly: **87.6%–88.2% accuracy** and **90.1%–90.7% F1**. This is a primary finding — a 52.4% feature reduction preserves competitive performance, validating the ANOVA F-Score selection strategy.

The **SVM (88.21%, F1=90.73%)** achieves the highest accuracy, consistent with the well-established strength of RBF-SVMs on tabular data. The **CNN+SVM hybrid (88.16%, F1=90.65%)** is nearly indistinguishable from standalone SVM, confirming that the CNN's 64-D latent representations carry at least as much discriminative information as the raw 20-D feature space — validating the hybrid concept. The **standalone CNN (87.60%, F1=90.13%)** slightly underperforms (~0.6% gap) but offers dramatically superior efficiency.

### 6.2 False Positive Rate Analysis

FPR determines alert-fatigue in operational IDS. Despite the SVM having the highest recall (84.84%), it generates the most false alarms:

| Model   | FPR   | False Alarms   | Precision | Operational Impact                                     |
|---------|-------|----------------|-----------|--------------------------------------------------------|
| CNN     | 2.88% | 1,615 / 56,000 | 98.40%    | Lowest FPR — best for low-alert-fatigue deployments    |
| CNN+SVM | 3.69% | 2,064 / 56,000 | 97.99%    | Intermediate — hybrid of CNN accuracy and SVM boundary |
| SVM     | 4.62% | 2,590 / 56,000 | 97.51%    | 60% more false alarms than CNN                         |

### 6.3 Inference Speed — Edge-Deployment Significance

**CNN: 0.34  $\mu$ s/sample** — well below inter-packet arrival times (~milliseconds in IoT). Confirms wire-speed classification capability for real-time edge deployment.

**SVM: 930  $\mu$ s/sample** — **2,737 $\times$  slower** than CNN. Root cause: SVM inference is  $O(n_{\text{support\_vectors}})$  per sample; with 3,077 KB of stored SVs the evaluation is prohibitively expensive on IoT microcontrollers.

**Key insight:** The CNN+SVM hybrid does NOT inherit the CNN's inference speed — the SVM kernel evaluation dominates. For real-time IoT edge deployment, the **standalone CNN (47.6 KB, 0.34  $\mu$ s/sample)** is the clear practical choice.

### 6.4 Feature Reduction Observations

Three distinct regions in the sweep: **(i) k=10–20:** sharp gains (+3.5% accuracy) — each added feature is informative; **(ii) k=20–30:** gradual plateau — diminishing returns; **(iii) k=30–35:** near-zero gain with slight degradation. k=20 is the optimal inflection point — 52.4% reduction with reasonable performance. Practically: 80 bytes/flow (20  $\times$  float32) vs. 168 bytes (42  $\times$  float32) buffer at the edge.

### 6.5 Methodological Comparison with Reference Papers

**Phalaagae et al. (IEEE Access 2025):** CNN-LSTM + Attention requires  $O(T \times H^2)$  recurrent computation and  $O(T^2)$  attention score matrix per flow window. Our approach omits both — reducing to a single MLP forward pass. Our preliminary result (88.2%, 90.7% F1) establishes a **complexity floor**: the reference paper's added complexity must exceed this baseline on matched datasets.

**Edge-AI IDS (IEEE ICERECT 2025):** Shares our motivation of proximity-to-device deployment. We provide a concrete lower-bound: a **47.6 KB, 3,457-parameter model** achieving 88.2% accuracy at **0.34  $\mu$ s/sample** — the first quantified complexity floor for this class of lightweight edge IDS on UNSW-NB15.

### 6.6 Limitations and Planned Next Steps

- No hyperparameter optimisation (grid search or Bayesian Multiclass attack categorisation (9 categories).
- Single pre-defined train/test split; no k-fold cross-validation hyperparameter tuning (grid search for C,  $\gamma$ , hidden layer dimensions).
- Binary classification only; nine attack categories merged into one class. CICIoT2023 (recent, IoT-specific, multi-class dataset).
- Timings on laptop CPU; not representative of IoT hardware (Raspberry Pi 4 / ESP32 / ARM Cortex-M).
- UNSW-NB15 (2015) may not capture recent IoT attack patterns — we are benchmarking on Raspberry Pi 4 / ESP32 for realistic latency.
- MLP used instead of true 1-D CNN — local receptive field and 1-D convolutional implementation for spatial feature extraction.

---

## References

- [1] N. Moustafa and J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems, in *Proc. IEEE MilCIS*, 2015.
  - [2] P. Phalaagae, A. M. Zungeru, A. Yahya, B. Sigweni and S. Rajalakshmi, A Hybrid CNN-LSTM Model With Attention Mechanism for Improved Intrusion Detection in Wireless IoT Sensor Networks, *IEEE Access*, 2025.
  - [3] Edge-AI Enabled Hybrid Deep Learning Framework for Botnet Intrusion Detection in Modern IoT-Driven Cyber Ecosystems, *IEEE ICERECT*, 2025.
  - [4] F. Pedregosa et al., Scikit-learn: Machine Learning in Python, *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
  - [5] V. N. Vapnik, *The Nature of Statistical Learning Theory*, Springer, 1995.
- 

All code, results, and models available at: [github.com/boringGuru347/DN-Project](https://github.com/boringGuru347/DN-Project)